RV College of Engineering®

Unit II –
Solving Problems by Searching

Contents -

1. Solving Problems by Searching

1.1 Problem-Solving Agents

•Definition of problem-solving agents
•Search problems and solutions
•Formulating problems

2. Example Problems

2.1 Grid World Problems

•Vacuum World
•Sokoban Puzzle

2.2 Real-World Problems

•Route-Finding Problems

3. Search Algorithms

3.1 Uninformed Search Strategies

•Breadth-First Search (BFS)
•Depth-First Search (DFS)

3.2 Informed (Heuristic) Search Strategies

•A* Search Algorithm

Introduction to Problem Solving

- **Problem solving** in Artificial Intelligence is the process of finding a sequence of actions that leads from an **initial state** to a **goal state**.
- An AI system does not immediately know the solution. Instead, it **searches through possible solutions**.

Example 1: Route Finding
Finding the shortest path between two cities.
Example:
Chennai → Bangalore
Possible routes:
- Chennai → Vellore → Bangalore
- Chennai → Krishnagiri → Bangalore

The agent searches for the **best route**.

Example 2: Robot Navigation
A robot moves in a room to reach a target location.
Goal:
Robot reaches destination without hitting obstacles.

What is a Problem in AI?

A **problem** is defined by:

- Where we start
- What actions are possible
- What goal we want
- How much each step costs

Basic Structure of a Problem

Every problem includes:

1. Initial State
2. Possible Actions
3. Goal State
4. Path Cost

Example: Maze Problem

Initial State:

Agent at entrance.

Actions:

Move left, right, up, down.

Goal:

Reach exit.

Cost:

Number of steps taken.

- Any problem solving agent follows this path

Goal formulation → Problem formulation → Search → Execution.

Goal formulation

- Goal formulation is the process of deciding what objective the agent wants to achieve.
- It defines the desired outcome and helps limit unnecessary actions.

Problem Formulation

- Problem formulation is the process of describing the problem in a structured way by identifying states, actions, and the goal.

- The agent creates a simplified model of the environment to understand how to reach the goal.

Initial State	Starting point
Possible Actions	Allowed moves
Goal Test	Checks if goal reached
Path Cost	Total cost of path

Search

- Search is the process of exploring possible sequences of actions to find a path from the initial state to the goal.
- The agent tries different possible paths until it finds one that reaches the goal.

Execution

- Execution is the process of carrying out the actions in the selected solution.
- Once the best path is found, the agent performs the actions step by step.

Example -Romania Map Problem & Problem-Solving Process

An agent is traveling in **Romania** and wants to reach **Bucharest** from **Arad**. From Arad, there are three possible roads leading to **Sibiu**, **Timisoara**, and **Zerind**, but the agent does not know which path leads to the goal without additional information.
If the environment is **unknown**, the agent must choose actions randomly.

- However, if the agent has access to information such as a **map**, it can solve the problem using a structured **four-phase problem-solving process**.

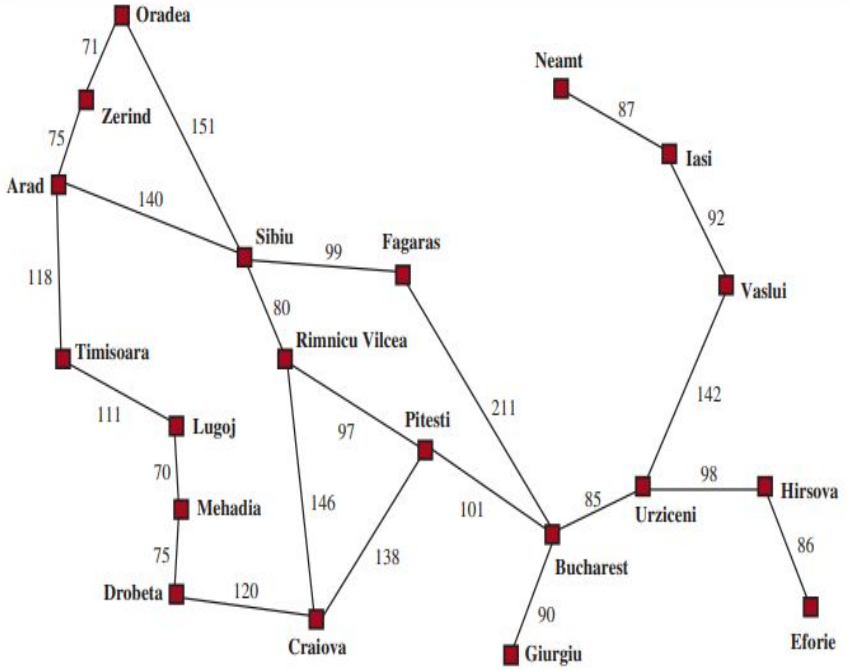


Figure 3.1 A simplified road map of part of Romania, with road distances in miles.

Four Phases of Problem Solving

1. Goal Formulation

The agent first decides **what it wants to achieve**.

Example:

Goal → Reach **Bucharest**

Goals help limit unnecessary actions and guide behavior.

2. Problem Formulation

The agent creates a simplified **model of the world**, including:

- States → Cities
- Actions → Travel from one city to another
- Only relevant information is considered (current city).

Example:

Current city = Arad

Actions = Travel to neighboring cities

Goal = Reach Bucharest

3. Search

The agent **simulates different paths** in the model until it finds a path that reaches the goal.

Example Solution Path:

Arad → Sibiu → Fagaras → Bucharest

4. Execution

After finding a solution, the agent **follows the steps** in the real world.

Example:

Travel from Arad to Sibiu, then to Fagaras, and finally to Bucharest.

Open-Loop System

- Used in **fully observable and deterministic environments**
- The agent follows a fixed sequence of actions
- No need to monitor environment while executing
- Example: Follow planned route without checking surroundings.

Closed-Loop System

- Used when the environment is **uncertain or partially observable**
- Agent continuously monitors percepts
- Adjusts actions if unexpected events occur
- Example: Changing route if a road is closed.

Search problems and solutions

A **search problem** consists of:

- State space (all possible states)
- Initial state (starting point)
- Goal state (desired outcome)
- Actions (possible moves)
- Transition model (result of actions)
- Action cost (cost of moves)

A **solution** is a path from the initial state to a goal state, and an **optimal solution** has the lowest cost

State space (all possible states)

- **Definition:**
  The **state space** is the set of **all possible states** that the environment can be in. Each state represents a situation in the environment.
- **Example (Romania Map):**
  State space = All cities such as:

1. Arad
2. Sibiu
3. Zerind
4. Timisoara
5. Bucharest

Initial state (starting point)

- **Definition:**
  The **initial state** is the **starting state** of the agent.
- **Example:**
  Initial state → **Arad**
- The agent begins solving the problem from here.

Goal state (desired outcome)

Definition:

A goal state is the state that satisfies the desired objective.

There may be:

- One goal state
- Multiple goal states
- A condition-based goal

Types of Goal States

1.Single Goal State

Example:

Goal → Reach **Bucharest**

2.Multiple Goal States

Example:

Goal → Reach **any airport**

3.Property-Based Goal

Goal defined using a condition.

Example (Vacuum World):

Goal → **No dirt anywhere**

IS-GOAL Method

Used to check:

Is the current state a goal state?

Actions (possible moves)

Actions are the **possible moves** available to the agent in a state.
Represented as:
ACTIONS(s)
Meaning:
Actions available in state **s**.

- The actions available to the agent. Given a state s, ACTIONS(s) returns a finite2 set of Action actions that can be executed in s. We say that each of these actions is applicable in s.
- Applicable An example: *ACTIONS(Arad) = {ToSibiu,ToTimisoara,ToZerind}.*

Transition model (result of actions)

The **transition model** describes the **result of performing an action**.

Represented as:

RESULT(s, a)

Meaning:

Result of applying action **a** in state **s**.

- A transition model, which describes what each action does. RESULT(s, a) returns the Transition model state that results from doing action a in state s. For example, *RESULT(Arad, ToZerind)* = *Zerind* .

Action cost (cost of moves)

The **action cost function** gives the **numerical cost** of performing an action.

Represented as:

c(s, a, s′)

Meaning:

Cost of moving from state **s** to **s′** using action **a**

- An action cost function, denoted by ACTION-COST(s,a, s′) when we are programming Action cost function or c(s,a, s′) when we are doing math, that gives the numeric cost of applying action a in state s to reach state s′ .

Path

Definition:
A path is a sequence of actions that connects states.

Example Path
Arad → Sibiu → Fagaras → Bucharest
This is a path from start to goal.

Path Cost Rule
Total path cost = **Sum of all action costs**
Example:
Arad → Sibiu = 140
Sibiu → Fagaras = 99
Fagaras → Bucharest = 211
Total cost = 450

Solution

Definition:

A solution is a path that leads from the initial state to a goal state.

Example

Solution path:

Arad → Sibiu → Fagaras → Bucharest

Optimal Solution

Definition:
An optimal solution is the solution with the lowest path cost.

Example
Two paths:
Path 1 → Cost = 450
Path 2 → Cost = 390
Optimal solution → Path 2

Important Assumption:
Action costs are **positive** and **additive**.

State Space as a Graph

The state space can be represented as a **graph**.

Graph Representation
Vertices (Nodes) → States
Edges (Links) → Actions

Example (Romania Map)
Cities → Nodes
Roads → Edges
Each road allows:
Travel in both directions
Two actions (forward and backward)

Formulating Problems

- Problem formulation means creating a simplified model of a real-world problem so that it can be solved using search techniques.
- Real-world problems contain too many details, so we simplify them using abstraction.
- In the given example --The Bucharest route problem is a model, not the real world.
- The description of traveling from one city to another (like Arad) is only a simplified mathematical model.
- It does not include every real-world detail.

- **A model is an abstract representation of reality.**
  - Instead of representing everything, the model includes **only important details** needed to solve the problem.
  - This simplified description is called an **abstract model**.
- **Real-world travel contains many details.**
  In an actual journey, the state of the world includes many factors such as:
  - Traveling companions
  - Music or radio program
  - Scenic views
  - Presence of police
  - Distance to rest stops
  - Road condition
  - Weather
  - Traffic

Abstraction

- Abstraction is the process of removing unnecessary details from a representation.
- It keeps only the important information needed to solve the problem.
- This helps make complex real-world problems simpler to handle.

Importance of the Right Level of Detail

- A good problem formulation must have the correct amount of detail.
- Too much detail makes the problem very complex.
- Too little detail makes the problem incorrect or incomplete.

Example of Too Much Detail

If actions were described in very small steps like:

Move right foot forward 1 cm
Turn steering wheel 1 degree

Then solving the problem would become extremely slow.
The agent might **never reach the destination**, such as Bucharest.

Level of Abstraction

Abstract States Represent Many Real States

- An abstract state represents many possible real-world situations.
- Example:
  - Abstract state: "In Arad"
  - Real states may include:
    - Different weather
    - Radio on/off
    - Different passengers

All these are treated as one abstract state.

Abstract Actions Represent Many Real Actions

- Abstract actions represent groups of detailed actions.
- Example:

  Abstract action:
  Drive from Arad to Sibiu
  Detailed actions:
  Start car
  Accelerate
  Turn steering wheel
  Adjust speed

These detailed steps are grouped into one simple action.

Abstract Solutions Represent Many Real Solutions

- A solution in the abstract model corresponds to many possible real-world solutions.
- Example:

  Abstract solution:
  Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest
  Real-world variations:
  Radio ON or OFF
  Different speeds
  Different stops
- All still follow the **same route**.

Valid and Useful – Abstract Solution

Validity of Abstraction -- What Makes an Abstraction Valid

- An abstraction is **valid** if: Every abstract solution can be converted into a **real-world solution**.

Example:
If a path exists from:
Arad → Sibiu
Then there must be a real road connecting them.
If no real path exists → Abstraction is invalid.

Usefulness of Abstraction - What Makes an Abstraction Useful

- An abstraction is **useful** if: Each abstract action is **easy to perform** in real life. It reduces complexity without losing correctness.

Example:
Action:
Drive from Arad → Sibiu
This is easy for a normal driver to perform.

Example Problems

- A standardized problem is a problem designed to demonstrate or test problem-solving methods.
- Examples of Standardized Problems
  - 8-Puzzle problem
  - Vacuum world problem
  - Route-finding problem (Romania map)
  - Sudoku
  - Chess problems

- A real-world problem is a problem that occurs in actual practical situations, where solutions are used in daily life.
- Examples of Real-World Problems
  - Robot navigation
  - Self-driving cars
  - GPS route planning
  - Medical diagnosis
  - Warehouse automation
  - Delivery systems

A **Sokoban puzzle** is a **grid-based game** where:

- An **agent (player)** pushes **boxes**
- Boxes must be moved to **designated storage locations**
- Only **one box per cell**
- Boxes **can be pushed but not pulled**

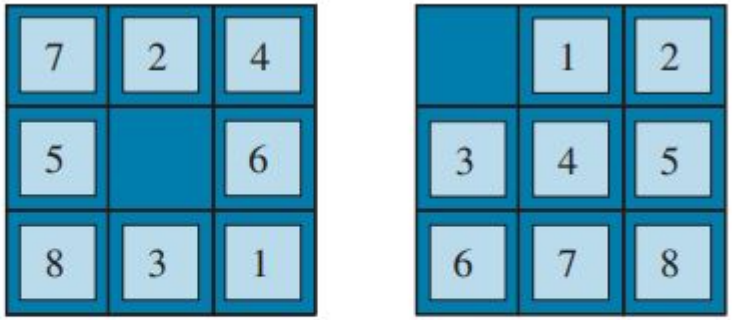


Figure 3.3 A typical instance of the 8-puzzle.

State Space Size

Given:

n = non-obstacle cells

b = number of boxes

Number of possible states:

$$\frac{n \times n!}{b!\,(n-b)!}$$

Example:

- 8×8 grid
- 12 boxes

Results in:

Over *200 trillion states*

That's why **efficient search algorithms** are required.

- States: A state description specifies the location of each of the tiles.
- Initial state: Any state can be designated as the initial state. Note that a parity property partitions the state space—any given goal can be reached from exactly half of the possible initial states (see Exercise 3.PART).
- Actions: While in the physical world it is a tile that slides, the simplest way of describing an action is to think of the blank space moving Left, Right, Up, or Down. If the blank is at an edge or corner then not all actions will be applicable.
- Transition model: Maps a state and action to a resulting state; for example, if we apply Left to the start state in Figure 3.3, the resulting state has the 5 and the blank switched.
- Goal state: Although any state could be the goal, we typically specify a state with the numbers in order, as in Figure 3.3.
- Action cost: Each action costs 1

Vacuum world problem

The **Vacuum Cleaner problem** is a classic example.
In the **two-cell vacuum world**, there are:

- **2 cells**
- Agent located in one cell
- Each cell may be:
  - Clean
  - Dirty

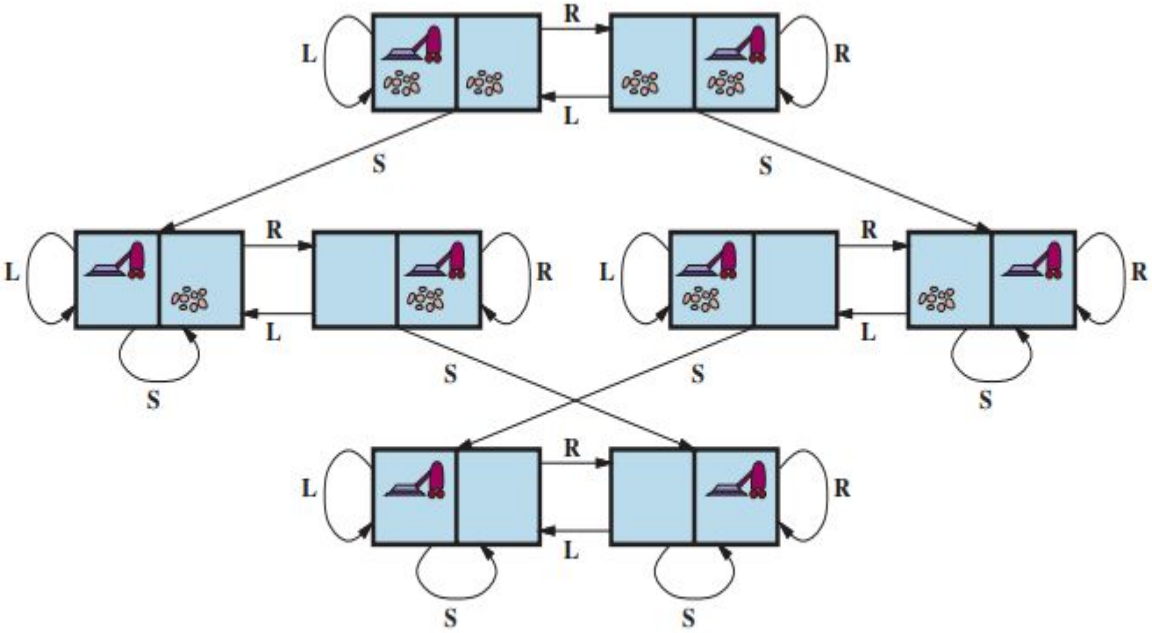


Figure 3.2 The state-space graph for the two-cell vacuum world. There are 8 states and three actions for each state: L = *Left*, R = *Right*, S = *Suck*.

- States: A state of the world says which objects are in which cells. For the vacuum world, the objects are the agent and any dirt. In the simple two-cell version, the agent can be in either of the two cells, and each cell can either contain dirt or not, so there are $2 \cdot 2 \cdot 2 = 8$ states (see Figure 3.2). In general, a vacuum environment with n cells has $n \cdot 2^n$ states.

• Initial state: Any state can be designated as the initial state.

• Actions: In the two-cell world we defined three actions: Suck, move Left, and move Right. In a two-dimensional multi-cell world we need more movement actions. We could add Upward and Downward, giving us four absolute movement actions, or we could switch to egocentric actions, defined relative to the viewpoint of the agent—for example, Forward, Backward, TurnRight, and TurnLeft.

• Transition model: Suck removes any dirt from the agent's cell; Forward moves the agent ahead one cell in the direction it is facing, unless it hits a wall, in which case the action has no effect. Backward moves the agent in the opposite direction, while TurnRight and TurnLeft change the direction it is facing by 90◦ .

• Goal states: The states in which every cell is clean.

• Action cost: Each action costs 1.

Real-World Problems

ROUTE-FINDING PROBLEM - involves moving from one location to another using paths (edges) between places.

• States: Each state obviously includes a location (e.g., an airport) and the current time. Furthermore, because the cost of an action (a flight segment) may depend on previous segments, their fare bases, and their status as domestic or international, the state must record extra information about these "historical" aspects.

• Initial state: The user's home airport.

• Actions: Take any flight from the current location, in any seat class, leaving after the current time, leaving enough time for within-airport transfer if needed.

• Transition model: The state resulting from taking a flight will have the flight's destination as the new location and the flight's arrival time as the new time.

• Goal state: A destination city. Sometimes the goal can be more complex, such as "arrive at the destination on a nonstop flight."

• Action cost: A combination of monetary cost, waiting time, flight time, customs and immigration procedures, seat quality, time of day, type of airplane, frequent-flyer reward points, and so on.

Commercial Travel Advice Systems

- These systems use **route-planning algorithms** similar to airline planning problems.
- **Key Points:**
- Handle complex airline fare rules.
- Must manage unexpected events like:
  - Flight delays
  - Missed connections
- Advanced systems include **backup (contingency) plans**.

- A **touring problem** involves visiting **multiple locations**, not just one destination.
- **Traveling Salesperson Problem (TSP):** Visit every city **exactly once**.
- Return to starting point.
- Goal:
  Find a route with cost less than C, or
  Find the **minimum-cost route**.
- **Applications:**

School bus route optimization
Delivery vehicle routing
Reducing traffic and pollution
Factory machine movements

VLSI Layout Problem (Chip Design)

- This problem involves arranging **millions of components** on a computer chip.
- **Two Main Steps:**

✔ **Cell Layout**

Group components into cells.
Place cells without overlapping.
Leave space for wiring.

✔ **Channel Routing**

Find paths for wires between cells.

- **Goal:**

Minimize chip size
Reduce delays
Improve manufacturing quality

Robot Navigation

- Robot navigation extends route-finding problems.
- **Key Points:**
- Robots create their own paths (not fixed roads).
- Movement space can be:
  - 2D (simple robots)
  - Multi-dimensional (robots with arms/legs)
- **Challenges:**

1. Sensor errors
2. Motor control errors
3. Partial knowledge of environment
4. Moving obstacles or other agents

Automatic Assembly Sequencing

- Used in **robotic manufacturing**.
- **Goal:**

Find the correct order to assemble parts.

- **Problem:**

Wrong order may make assembly impossible without undoing work.

- **Importance:**

Reduces manual labor

Saves time and cost

Requires careful feasibility checking

Protein Design

- A complex search problem in **bioinformatics**.
- **Goal:**

Find a sequence of amino acids.

Ensure it folds into the correct **3D protein structure**.

Used to design proteins that can **treat diseases**.

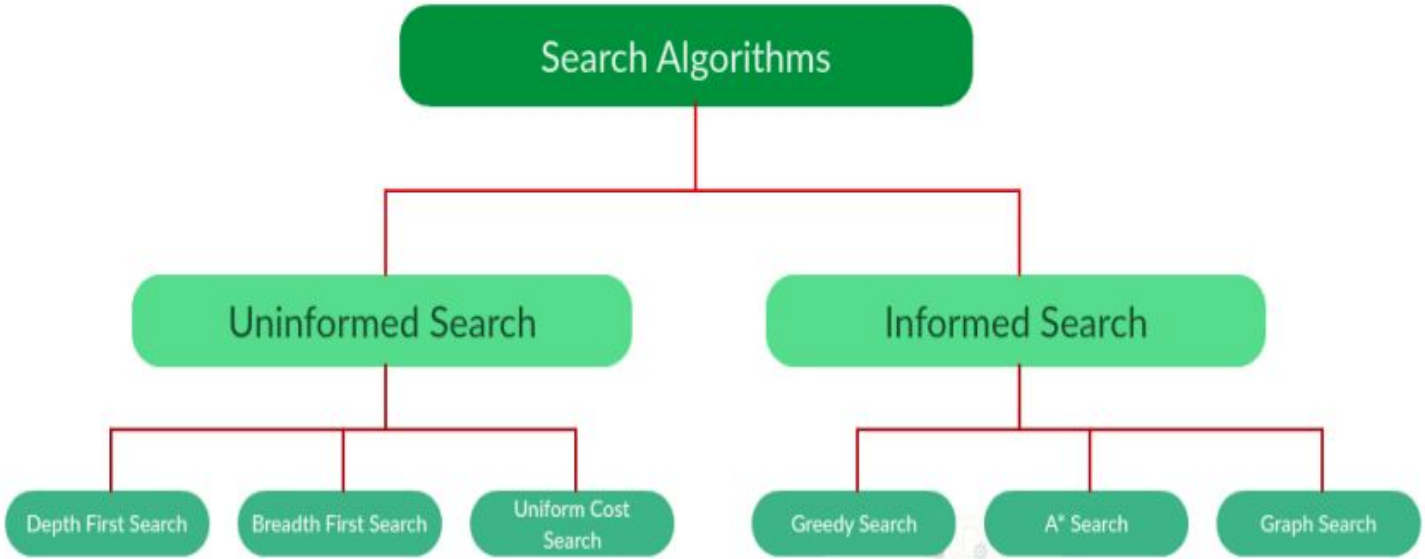
Search Algorithms

- Artificial Intelligence is the study of building agents that act rationally. Most of the time, these agents perform some kind of search algorithm in the background in order to achieve their tasks.
- A search problem consists of:

o A State Space. Set of all possible states where you can be.
o A Start State. The state from where the search begins.
o A Goal State. A function that looks at the current state returns whether or not it is the goal state.

- The Solution to a search problem is a sequence of actions, called the plan that transforms the start state to the goal state.
- This plan is achieved through search algorithms. Types of search algorithms: There are far too many powerful search algorithms out there to fit in a single article. Instead, this article will discuss six of the fundamental search algorithms, divided into two categories, as shown below.

Types of search algorithms:



Uninformed Search Algorithms:

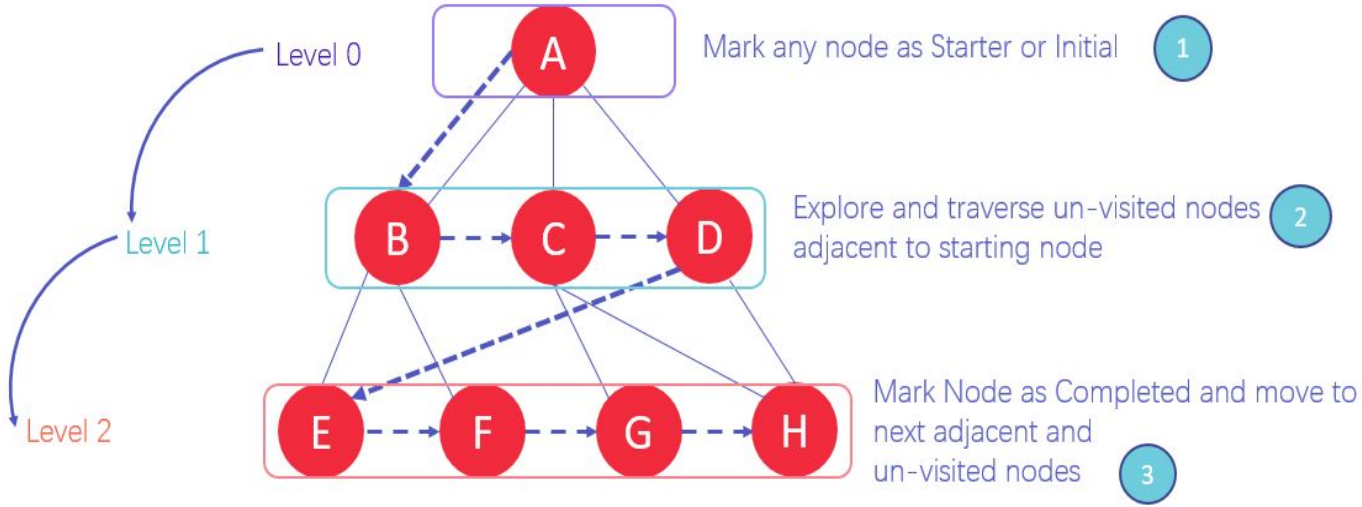
- The search algorithms in this section have no additional information on the goal node other than the one provided in the problem definition.
- The plans to reach the goal state from the start state differ only by the order and/or length of actions.
- Uninformed search is also called Blind search.
- These algorithms can only generate the successors and differentiate between the goal state and non goal state.

Breadth-first search (BFS)

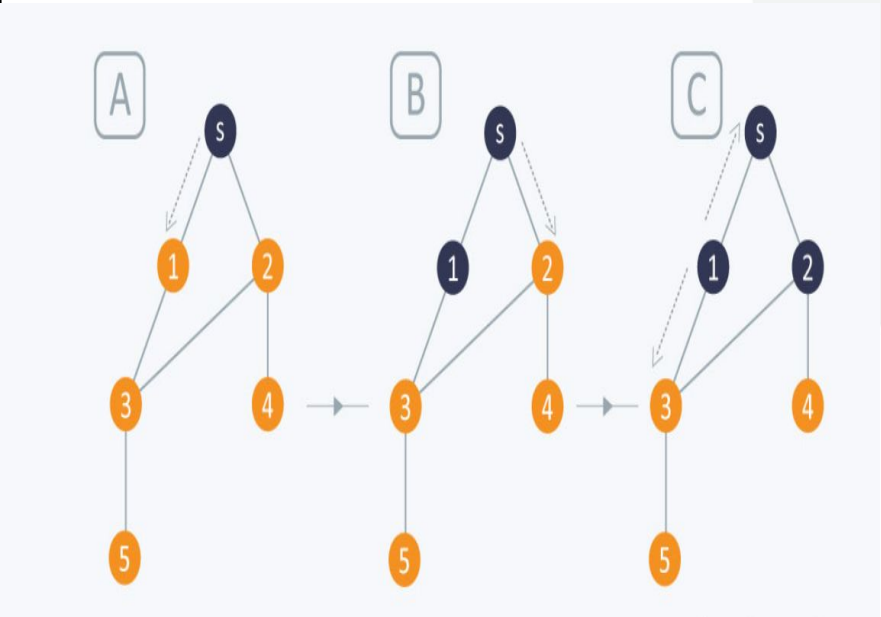

```
function BREADTH-FIRST-SEARCH(problem) returns a solution node or failure
  node ← NODE(problem.INITIAL)
  if problem.IS-GOAL(node.STATE) then return node
  frontier ← a FIFO queue, with node as an element
  reached ← {problem.INITIAL}
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    for each child in EXPAND(problem, node) do
      s ← child.STATE
      if problem.IS-GOAL(s) then return child
      if s is not in reached then
        add s to reached
        add child to frontier
  return failure

function UNIFORM-COST-SEARCH(problem) returns a solution node, or failure
  return BEST-FIRST-SEARCH(problem, PATH-COST)
```

CONCEPT DIAGRAM



Example



First iteration

s will be popped from the queue
Neighbors of s i.e. 1 and 2 will be traversed
1 and 2, which have not been traversed earlier, are traversed. They will be:
- Pushed in the queue
- 1 and 2 will be marked as visited

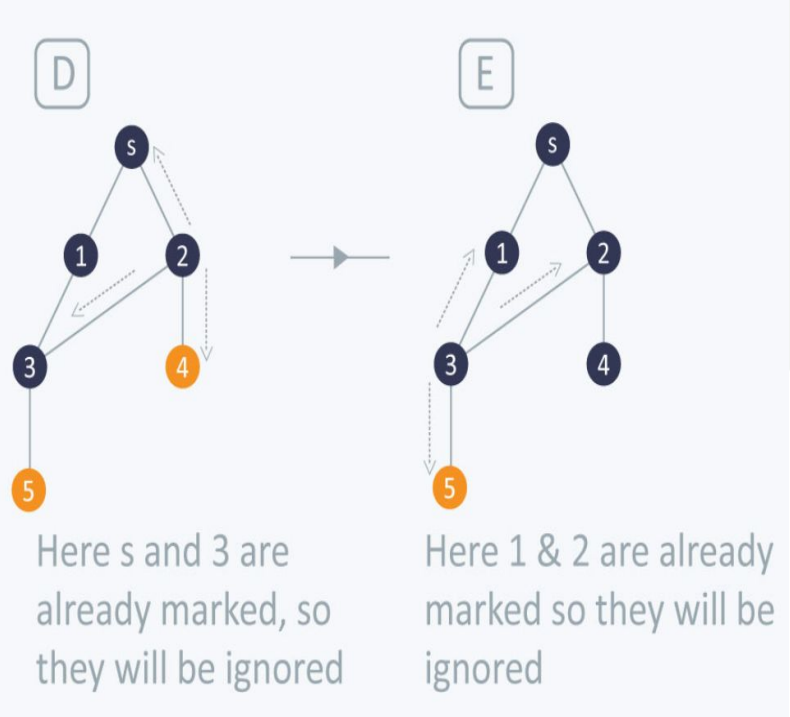
Second iteration

1 is popped from the queue
Neighbors of 1 i.e. s and 3 are traversed
s is ignored because it is marked as 'visited'
3, which has not been traversed earlier, is traversed. It is:
- Pushed in the queue
- Marked as visited

Third iteration

2 is popped from the queue
Neighbors of 2 i.e. s, 3, and 4 are traversed
3 and s are ignored because they are marked as 'visited'
4, which has not been traversed earlier, is traversed. It is:
- Pushed in the queue
- Marked as visited

Here s will be ignored

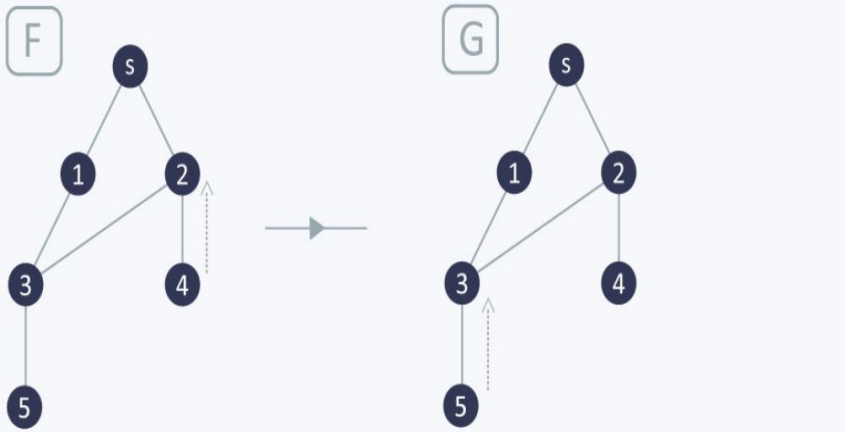


Third iteration

- 2 is popped from the queue
- Neighbors of 2 i.e. s, 3, and 4 are traversed
- 3 and s are ignored because they are marked as 'visited'
- 4, which has not been traversed earlier, is traversed. It is:
  - Pushed in the queue
  - Marked as visited

Fourth iteration

- 3 is popped from the queue
- Neighbors of 3 i.e. 1, 2, and 5 are traversed
- 1 and 2 are ignored because they are marked as 'visited'
- 5, which has not been traversed earlier, is traversed. It is:
  - Pushed in the queue
  - Marked as visited



Fifth iteration

- 4 will be popped from the queue
- Neighbors of 4 i.e. 2 is traversed
- 2 is ignored because it is already marked as 'visited'

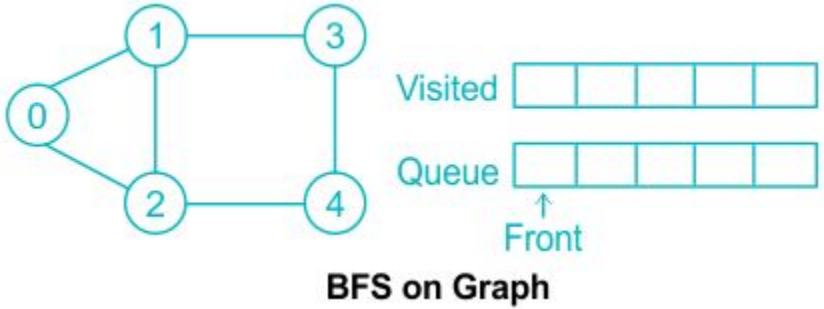
Sixth iteration

- 5 is popped from the queue
- Neighbors of 5 i.e. 3 is traversed
- 3 is ignored because it is already marked as 'visited'

The queue is empty and it comes out of the loop. All the nodes have been traversed by using BFS.

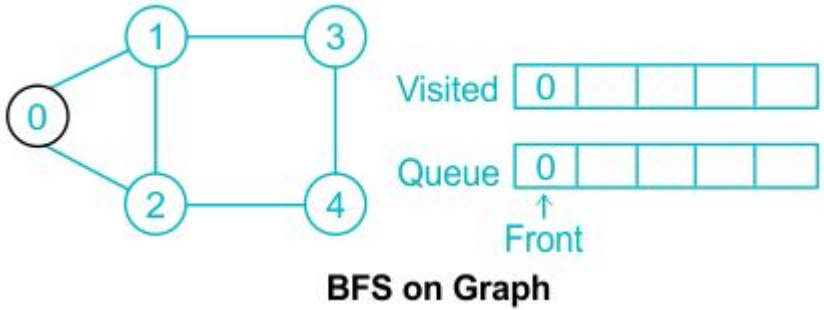
If all the edges in a graph are of the same weight, then BFS can also be used to find the minimum distance between the nodes in a graph.

Step 1: Initially the queue and visited arrays are empty.



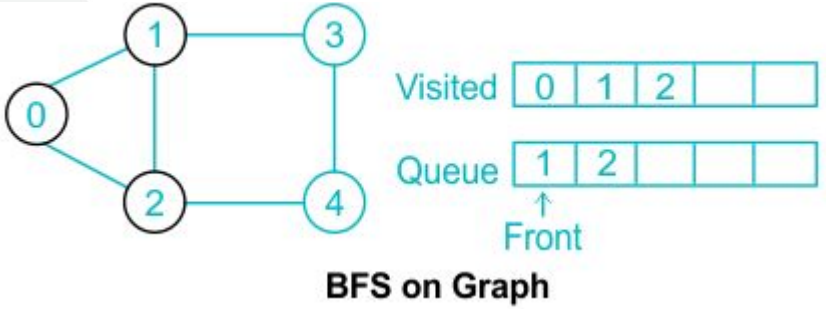
BFS on Graph

Step 2: Push node 0 into the queue and mark it visited.



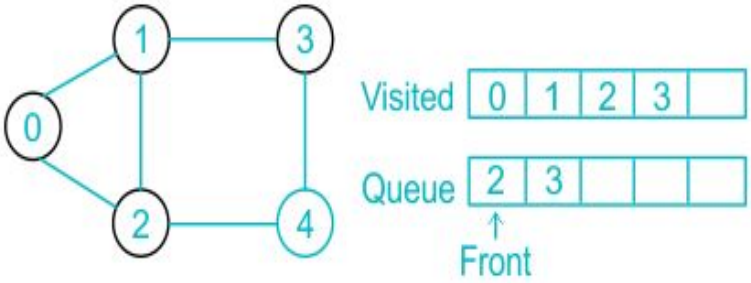
BFS on Graph

Step 3: Remove node 0 from the front of the queue and visit the unvisited neighburs and push them into the queue.



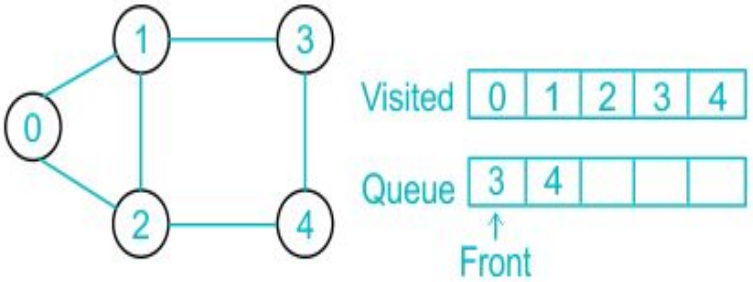
BFS on Graph

Step 4: Remove node 1 from the front of the queue and visit the unvisited neighbours and push them into the queue.



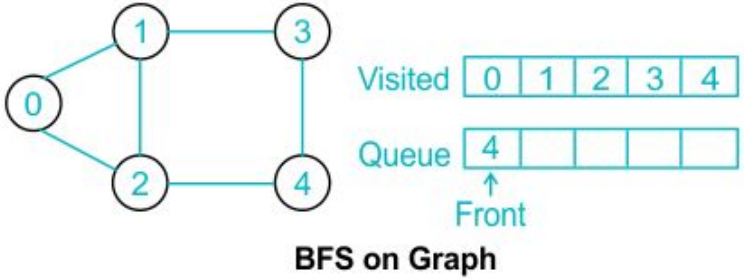
BFS on Graph

Step 5: Remove node 2 from the front of the queue and visit the unvisited neighbours and push them into the queue.



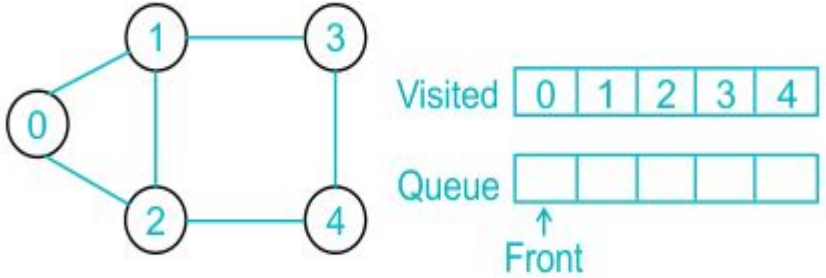
BFS on Graph

Step 6: Remove node 3 from the front of the queue and visit the unvisited neighbours and push them into the queue.
As we can see that every neighbour of node 3 is visited, so move to the next node that is in the front of the queue.



BFS on Graph

Step 7: Remove node 4 from the front of the queue and visit the unvisited neighbours and push them into the queue.
As we can see that every neighbour of node 4 is visited, so move to the next node that is in the front of the queue.



BFS on Graph

- Advantages of Breadth First Search

Finds the path of minimal length to the goal.

- Disadvantages of Breadth First Search

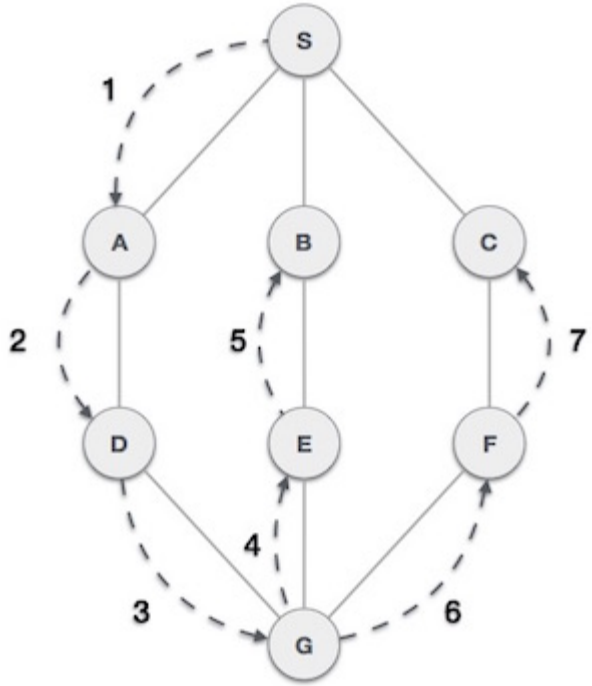
Requires the generation and storage of a tree whose size is exponential the the depth of the shallowest goal node

Depth-first search (DFS)

- Depth First Search (DFS) algorithm is a recursive algorithm for searching all the vertices of a graph or tree data structure. This algorithm traverses a graph in a depthward motion and uses a stack to remember to get the next vertex to start a search, when a dead end occurs in any iteration.

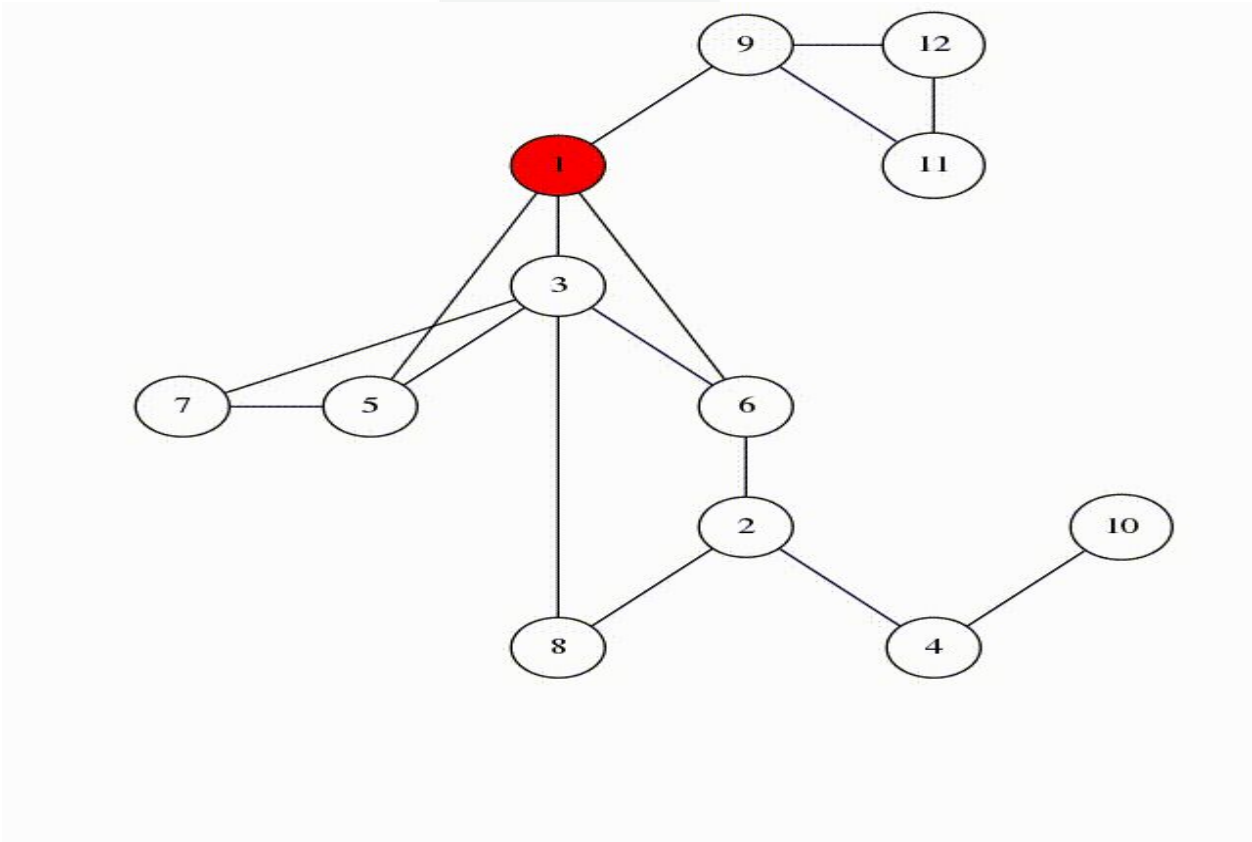
DFS Algorithm -

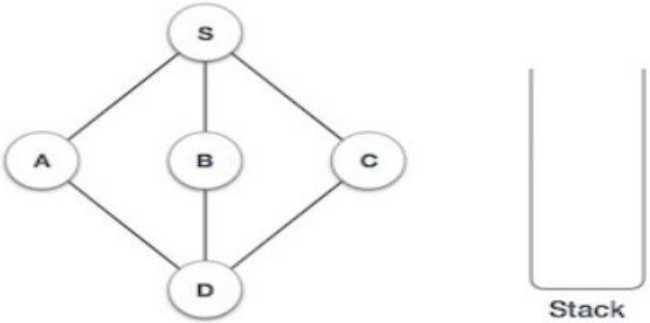
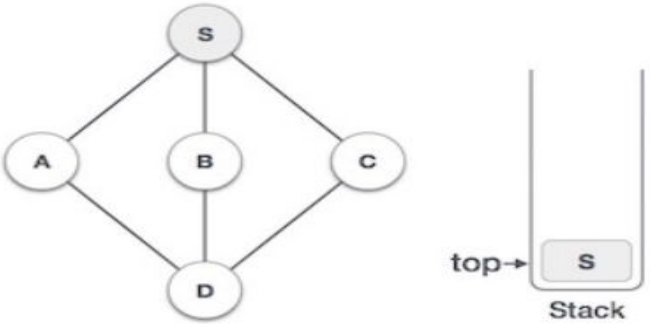
- Choose a starting vertex and mark it as visited.
- Visit the starting vertex and explore its adjacent vertices.
- For each unvisited adjacent vertex, recursively apply DFS.
- Continue exploring vertices until all reachable vertices are visited or no unvisited vertices remain.
- If all reachable vertices are visited, backtrack to the previous vertex and explore other branches.

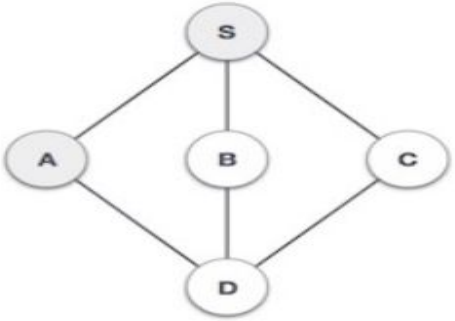
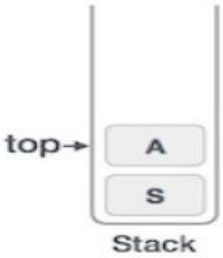
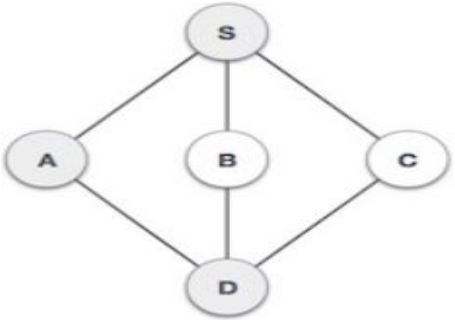
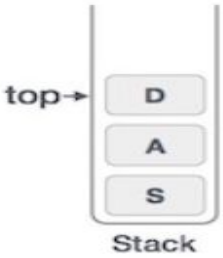


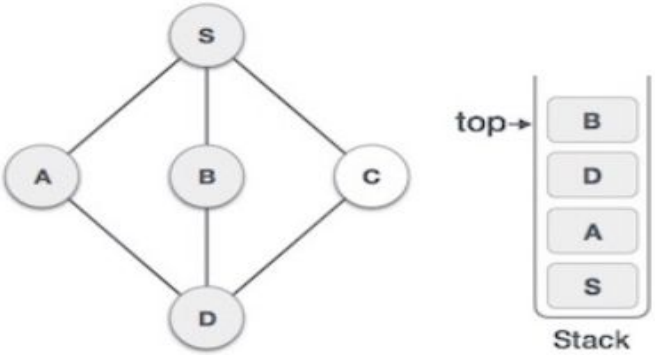
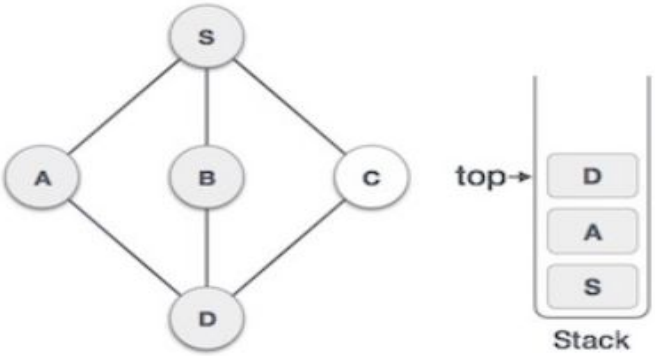
As in the example given above, DFS algorithm traverses from S to A to D to G to E to B first, then to F and lastly to C.

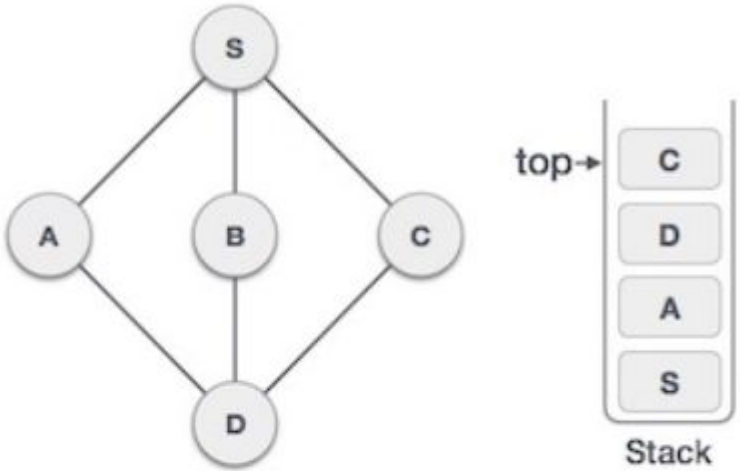
Example



Step	Traversal	Description
1		Initialize the stack.
2		Mark **S** as visited and put it onto the stack. Explore any unvisited adjacent node from **S**. We have three nodes and we can pick any of them. For this example, we shall take the node in an alphabetical order.

3			Mark **A** as visited and put it onto the stack. Explore any unvisited adjacent node from A. Both **S** and **D** are adjacent to **A** but we are concerned for unvisited nodes only.
4			Visit **D** and mark it as visited and put onto the stack. Here, we have **B** and **C** nodes, which are adjacent to **D** and both are unvisited. However, we shall again choose in an alphabetical order.

5		We choose **B**, mark it as visited and put onto the stack. Here **B** does not have any unvisited adjacent node. So, we pop **B** from the stack.
6		We check the stack top for return to the previous node and check if it has any unvisited nodes. Here, we find **D** to be on the top of the stack.

7		Only unvisited adjacent node is from **D** is **C** now. So we visit **C**, mark it as visited and put it onto the stack.

As **C** does not have any unvisited adjacent node so we keep popping the stack until we find a node that has an unvisited adjacent node. In this case, there's none and we keep popping until the stack is empty.

Informed Search Algorithms:

- Informed Search Strategy

• One that uses problem-specific knowledge beyond the definition of the problem itself

• This finds the solutions more efficiently than uninformed strategy

▪ For example :

• The search algorithm comprise of the knowledge like how far the goal is, path cost, how to reach to goal node, etc.

• This knowledge help agents to explore less to the search space and find more efficiently the goal node

- Heuristic is a function which is used in Informed Search, and it finds the most promising path.
- It takes the current state of the agent as its input and produces the estimation of how close agent is from the goal
- The heuristic method, might not always give the best solution, but it guaranteed to find a good solution in reasonable time
- Heuristic function estimates how close a state is to the goal - It is represented by h(n), and it calculates the cost of an optimal path between the pair of states
- The value of the heuristic function is always positive
- Heuristic function is given as:

$f(n) = h(n)$

Here, f(n) is heuristic cost, and h(n) is the estimated cost. Hence heuristic cost should be less than or equal to the estimated cost.

A* Search Algorithm

- A* search is the most commonly known form of best-first search
- It uses heuristic function h(n), and cost to reach the node ‘n’ from the start state g(n)
- A* search algorithm finds the shortest path through the search space using the heuristic function
- This search algorithm expands less search tree and provides optimal result faster
- In A* search algorithm, we use search heuristic as well as the cost to reach the node. Hence we can combine both costs as following, and this sum is called as a fitness number

A* Search Algorithm

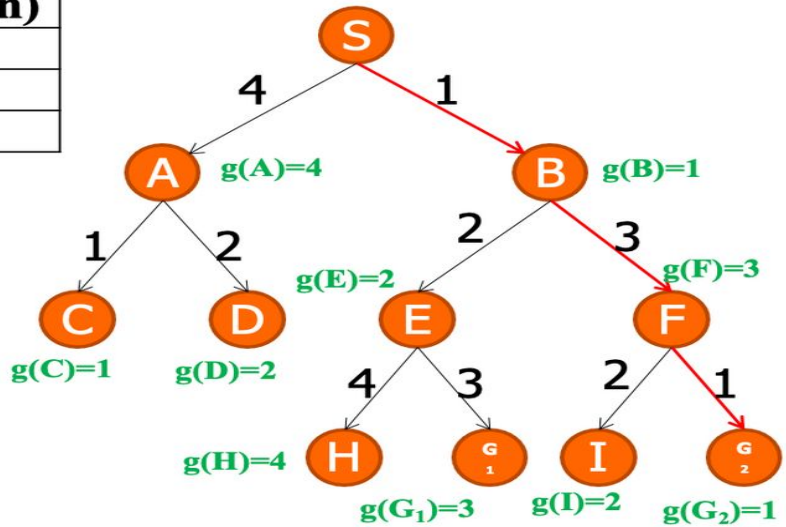
Step 1: Initialize: Initialize the open list with the start node and the closed list as empty. The open list contains nodes to be evaluated, while the closed list contains nodes that have already been evaluated.
Step 2: Select Node: Select the node with the lowest f value (f = g + h) from the open list and move it to the closed list.
Step 3: Check Goal: If the selected node is the goal node, reconstruct the path and return it as the solution.
Step 4: Expand Node: For each neighbor of the selected node, calculate its g, h, and f values. If the neighbor is not in the open list, add it. If it is already in the open list with a higher f value, update its f value and parent node.
Step 5: Repeat: Repeat steps 2 to 4 until the open list is empty or the goal node is found.

EXAMPLE on A* ALGORITHM (cont.)

EXAMPLE: Use A* search algorithm to find the solution.

Initial state: S, Goal state: G_1 or G_2

node	h(n)	node	h(n)	node	h(n)
A	11	D	8	H	7
B	5	E	4	I	3
C	9	F	2		

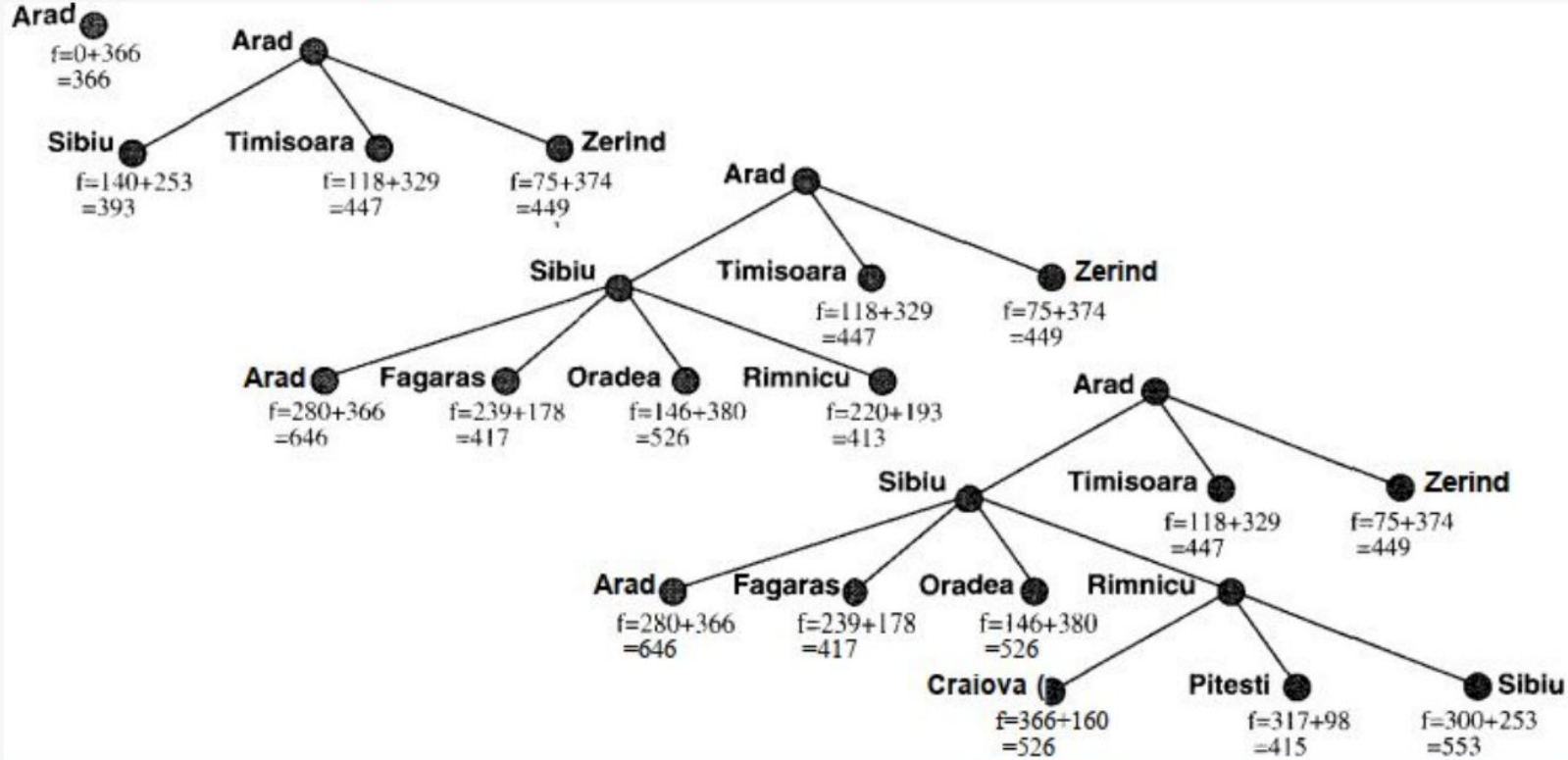


Solution:

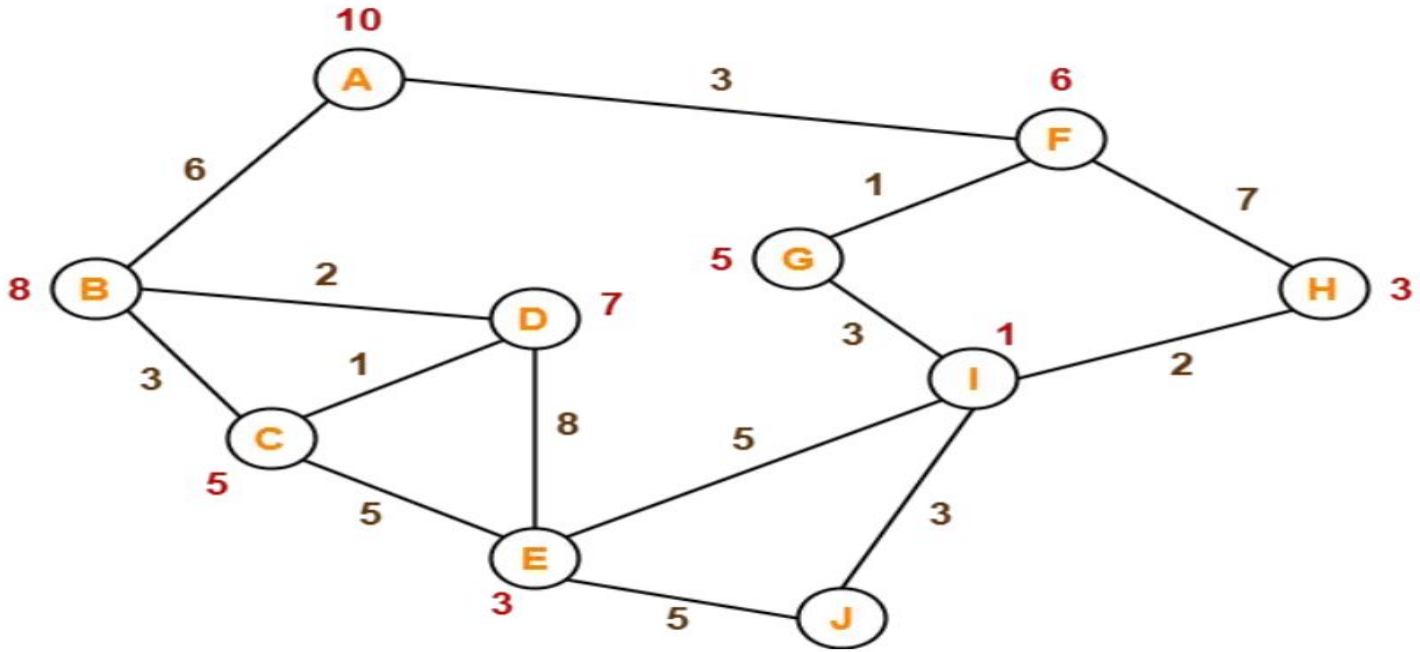
It.	Node expanded	Priority queue
0		S=0
1	S	A=15, B=6
2	B	E=7, F=6, A=15
3	F	I=9, G_2=5, E=7, A=15
4	G2	G_2

S , B , F , G_2

⇒ A* Search Algorithm



Example



Find the most cost-effective path to reach from start state A to final state J using A* Algorithm

Step-01:

A* Algorithm calculates f(B) and f(F).

f(B) = 6 + 8 = 14

f(F) = 3 + 6 = 9

Since f(F) < f(B), so it decides to go to node F.

Path- A → F

Step-02:

Node G and Node H can be reached from node F.

A* Algorithm calculates f(G) and f(H).

- f(G) = (3+1) + 5 = 9
- f(H) = (3+7) + 3 = 13

Since f(G) < f(H), so it decides to go to node G

Path- A → F → G

Step-03:

Node I can be reached from node G.

A* Algorithm calculates f(I).
f(I) = (3+1+3) + 1 = 8
It decides to go to node I.

Path- A → F → G → I

Step-04:

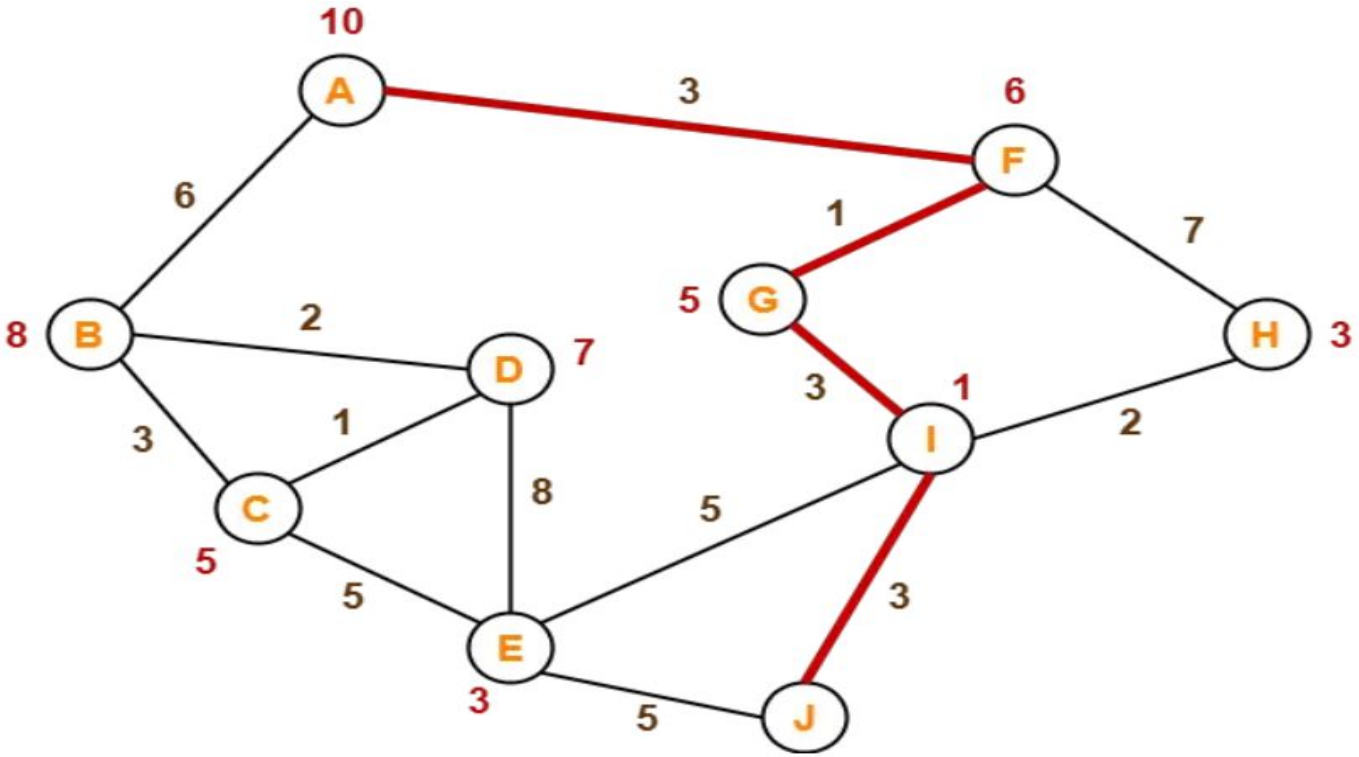
Node E, Node H and Node J can be reached from node I.

A* Algorithm calculates f(E), f(H) and f(J).

- f(E) = (3+1+3+5) + 3 = 15
- f(H) = (3+1+3+2) + 3 = 12
- f(J) = (3+1+3+3) + 0 = 10

Since f(J) is least, so it decides to go to node J.

Path- A → F → G → I → J


10
A
3
6
F
6
1
7
5
G
8
B
2
H
3
D
7
3
1
1
I
2
3
8
5
C
5
5
3
E
3
5
J